

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1
«розгортання середовища та порівняльний аналіз мовних моделей»
з дисципліни: «Обчислювальні методи»

Виконав: студент групи AI-244
Демешко Микита
Ігорович

Перевірив: ас. Баранов О.М.

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Виконано запити до двох моделей через ендпойнт `/api/generate`. Спочатку – просте фактичне питання, потім – задача на міркування (5 запусків для кожної моделі). Перед серією вимірів зроблено прогрівальний запит.

Таблиця 1 – Фактичне питання «Столиця України? Відповідай одним словом.»

Показник	qwen3:4b	llama3.2:3b
response	Київ	Київ
eval_count	2573	5
Час, с	256,58	5,29
Токенів/с	10,05	20,4

Таблиця 2 – Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5 / 5	3 / 5
eval_count	1074,4	186,2
Час, с	94,02	12,89

Типова помилка інструктивної моделі: відповідь «10 грн», отримана без урахування того, що різниця у 100 грн розподіляється між двома доданками.

Відповіді на питання (крок 6)

1. Куди пішли додаткові токени в міркувальній моделі?

Відповідь: Токени пішли на генерацію внутрішнього ланцюжка міркувань (thinking). Модель детально зіставляла мову запиту, правила орфографії, наявність диакритики в українській назві («Київ» чи «Киев») та багаторазово перевіряла саму себе перед формуванням остаточного виводу.

2. На яких типах задач міркувальні моделі дають перевагу?

Відповідь: Міркувальні моделі незамінні в логічних, алгоритмічних, математичних та багатоетапних задачах (де потрібен покроковий розв'язок і самоперевірка

проміжних обчислень). Їхнє використання не виправдане на простих фактологічних запитах, класифікації тексту чи задачах витягу даних, де потрібна мінімальна затримка, а довгий роздум лише безглуздо витрачає обчислювальні ресурси та час.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Відповідь: Інструктивні моделі навчаються методом класичного supervised fine-tuning (SFT) та RLHF одразу давати компакту цільову відповідь на інструкцію. Міркувальні моделі навчаються з використанням масштабного Reinforcement Learning (RL), де винагороджується побудова правильного й розгорнутого ланцюжка думок (Chain-of-Thought) із верифікацією та виправленням власних помилок перед фінальною відповіддю.

4. Чи означає більша кількість токенів, що модель «розумніша»?

Відповідь: Ні, більша кількість токенів свідчить лише про більший обсяг виконаної обчислювальної роботи у фазі генерації роздумів, а не про принципово вищий інтелект. Проте на складних задачах цей внутрішній контекст дає моделі змогу утримувати проміжні результати в пам'яті, що знижує ймовірність логічних збоїв.

5. Як пояснити різницю у швидкості генерації між моделями?

Відповідь: Llama3.2:3b має 3.21 млрд параметрів, тоді як qwen3:4b — понад 4 млрд параметрів. Швидкість інференсу на CPU безпосередньо залежить від пропускну здатності оперативної пам'яті: менша модель вимагає менше операцій перемноження матриць на один токен, тому Llama видає відчутно більший показник токенів за секунду (~20.4 проти ~10.05).

Висновок за завданням 1

Експериментальне порівняння інструктивної моделі Llama 3.2 (3B) та міркувальної моделі Qwen 3 (4B) продемонструвало ключовий інженерний компроміс між швидкістю інференсу та точністю розв'язання логічних задач:

1. На простому фактичному питанні обидві моделі дали правильну відповідь («Київ»), проте їхня поведінка кардинально відрізнялася. Llama 3.2 згенерувала відповідь практично миттєво (5 токенів, 5,29 с чистої генерації). Натомість Qwen 3 витратила понад 4 хвилини (2573 токени), генеруючи розлогий внутрішній

ланцюжок міркувань з надмірною самоперевіркою правил орфографії та мови. Це підтверджує, що для тривіальних фактологічних запитів використання міркувальних моделей є невиправданою перевитратою часу та ресурсів;

2. У задачі про ручку та зошит інструктивна Llama 3.2 показала нестабільний результат (лише 3 правильні відповіді з 5), схильючись до інтуїтивних помилок (50 грн, 90 грн) та збиваючись на суржик. Міркувальна Qwen 3 продемонструвала 100% точність (5 з 5): завдяки фазі роздумів вона щоразу будувала формальну систему рівнянь, підставляла змінні та проводила самоперевірку перед видачею фінального тексту.

Тобто маємо, що інструктивні моделі (типу Llama 3.2) доцільно обирати для діалогових систем, чат-ботів, класифікації та витягу фактів, де критичною є мінімальна затримка (TTFT) і висока швидкість генерації. А міркувальні моделі (типу Qwen 3) слід застосовувати в аналітичних пайплайнах, математичних розрахунках, генерації складного коду та задачах, де вартість логічної помилки значно вища за витрачений час на обчислення.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Виконано запити одного питання до двох моделей через python скрипт з різними інструкціями. Спочатку – надається інструкція «лаконічного технічного асистента», а далі інша інструкція просить «відповідати лише українською, та максимальний обсяг відповіді – два речення».

Таблиця 3 – Вплив system-повідомлення

Модель	System	Довжина відповіді, символів	Мова відповіді	Кількість речень	Інструкцію виконано?
llama3.2:3b	лаконічний асистент	1805	Українська (суржик,	12	Ні (відповідь занадто велика)

			вкраплення англійської)		
llama3.2:3b	українською, 2 речення	188	Українська (наявність дивних символів)	2	Частково (речень 2, але текст зіпсовано)
qwen3:4b	лаконічний асистент	207	Українська (єдине слово калька - "узліками")	2	Так
qwen3:4b	українською, 2 речення	351	Українська грамотна	2	Так

Таблиця 4: Витрати токенів

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	54	597	651
llama3.2:3b	українською, 2 речення	58	65	123
qwen3:4b	лаконічний асистент	57	2224	2281
qwen3:4b	українською, 2 речення	66	1120	1186

Відповіді на питання (крок 6)

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Відповідь: Обидві моделі формально вклалися у два речення, проте qwen3:4b зробила це значно якісніше та зв'язніше, тоді як llama3.2:3b згенерувала напівграмотну нісенітницю із незрозумілими символами.

2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?

Відповідь: Прямого переходу на іншу мову не відбулося, проте llama3.2:3b видавала текст суржиком і калькованими конструкціями, а також додала сторонній шрифт у другому запуску.

3. Як змінився `prompt_tokens` при зміні `system`-повідомлення і чому?

Відповідь: Показник `prompt_tokens` зріс (у Llama з 54 до 58, у Qwen з 57 до 66), оскільки інструкція «Відповідай лише українською, максимум двома реченнями» довша за символами й словами за «Ти лаконічний технічний асистент», тому вимагає більше токенів для токенизації вхідного контексту.

4. Чому в `qwen3:4b` значення `completion_tokens` значно більше, ніж у `Llama3.2:3b`, навіть при однаковій довжині видимої відповіді?

Відповідь: У `qwen3:4b` до `completion_tokens` входять не лише токени видимої фінальної відповіді, а й усі токени внутрішнього міркування, які модель породжує в процесі аналізу задачі перед видачею результату.

Відповіді на питання (крок 7)

1. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови?

Відповідь: Різні моделі мають різну архітектуру та глибину навчання з підкріпленням (RLHF/RL). `Llama3.2:3b` є чисто інструктивною малою моделлю, яка слабо контролює ліміти токенизації при генерації, тоді як `qwen3:4b` використовує міркувальні кроки для планування обсягу тексту перед генерацією відповіді.

2. Чим `system`-повідомлення відрізняється від `user`-повідомлення з точки зору моделі?

Відповідь: `System`-повідомлення обрамляється спеціальними службовими токенами ролі (наприклад, `<|im_start|>system`), які під час навчання моделі мали найвищу вагу для формування контексту поведінки. Проте при виникненні конфлікту між ролями модель може орієнтуватися на останній вхідний токен (`user`), якщо вага інструкції в системному промпті недостатньо сильна.

3. Чому міркувальна модель витрачає більше вихідних токенів при однаковій видимій довжині?

Відповідь: API повертає сукупну кількість згенерованих вихідних токенів за один запит. Для моделі типу `qwen3` це сума токенів у прихованому блоці міркувань

(thinking) та власне фінальної відповіді, через що лічильник `completion_tokens` виявляється в рази більшим.

4. Що станеться при суперечливих інструкціях у `system` і `user`?

Відповідь: Модель або спробує знайти компроміс між вимогами, або віддасть перевагу `user`-повідомленню через ефект близькості контексту (*recency bias*), або почне галюцинувати й порушить обидва обмеження.

5. Чому малі моделі гірше дотримуються інструкцій щодо мови?

Відповідь: Малі моделі (3–4 млрд параметрів) мають менший словниковий запас і обмежений обсяг неанглійських навчальних корпусів. Розуміння української семантики в них потребує активації ширших асоціативних зв'язків, які часто перетинаються з домінуючими англомовними або російськомовними патернами.

Висновок за завданням 2

System-повідомлення є простим способом загального спрямування тону та ролі моделі, проте воно не гарантує суворого детермінованого дотримання формату й мови (особливо на моделях розміром 3–4B). Самих лише текстових інструкцій недостатньо для продакшн-систем.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Виконано запити одного питання на моделі хмарно та локально. Використовуємо Gemini API Key для хмарного звернення.

Таблиця 5: Порівняння локальної та хмарної моделі

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	gemini-3.5-flash-lite
Час відповіді, с	16.46	1.94
<code>completion_tokens</code>	201	345
Довжина відповіді, символів	761	1144

Мова відповіді	Українська (ламаний суржик: «один-anOTHER», «референції»)	Українська (чиста, літературна технічна термінологія)
Фактичних помилок у відповіді	2 (твердження «елементи мають однаковий адрес» та «список не розташований в пам'яті під час виконання»)	0 (точний опис адресації, асимптотики $O(1)/O(N)$ та правил вибору)

Відповіді на питання (крок 5)

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Відповідь: Рівно нуль рядків у логіці виконання. Завдяки єдиному стандарту OpenAI-сумісного API весь перехід реалізований через конфігураційний словник, де змінюються лише два рядки: `base_url` та назва моделі `model` (разом із передачею ключа `api_key`).

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Відповідь: Відповідь хмарної моделі набагато точніша та інформативніша: вона чітко розклала структури за пам'яттю, асимптотичною складністю операцій доступу та вставки ($O(1)$ проти $O(N)$) і додала практичне правило вибору. Локальна модель видала плутані некоректні твердження на кшталт «елементи масиву мають однаковий адрес».

3. Чи були в локальній моделі фактичні помилки або проблеми з українською мовою?

Відповідь: Так, були грубі фактичні помилки (стверджувалося, що елементи зв'язного списку не розташовані в пам'яті під час виконання програми), а також сильні дефекти мови: русизми («зібрання»), транслітеровані кальки («референції») та спотворення слів англійськими вставками («один-anOTHER»).

4. Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит летить через інтернет?

Хмарна модель працює у дата-центрі на спеціалізованих тензорних прискорювачах (TPU/GPU) з гігантською пропускнуою здатністю пам'яті, що дає швидкість генерації понад 150–200 токенів за секунду. Локальна модель обчислюється на CPU персонального комп'ютера, де шина оперативної пам'яті DDR є вузьким місцем і обмежує швидкість до 10–20 токенів за секунду.

Відповіді на питання (крок 6)

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?

Відповідь: OpenAI першою вивела комерційні API для LLM на масовий ринок, створивши навколо своєї специфікації готову екосистему бібліотек, фреймворків (LangChain, LlamaIndex) та плагінів. Підтримка ідентичного контракту (/chat/completions) дозволяє іншим провайдерам та інструментам на зразок Ollama інтегруватися у вже існуючий софт клієнтів взагалі без переписування коду.

2. Я порівнював локальну модель на 3 млрд параметрів і хмарну на десятки/сотні млрд. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?

Відповідь: Кількість параметрів принципово визначає «місткість» знань і здатність до абстрактного узагальнення. Моделі 3B фізично бракує параметрів для збереження рідкісних мовних патернів української мови чи тонкощів теорії алгоритмів. Архітектура, якість датасету та вирівнювання важливі, але масштаб моделі залишається вирішальним фактором для глибини та точності виводу.

3. Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?

Відповідь: Мережева затримка (RTT) зазвичай складає лише 50–150 мс і впливає тільки на час початку генерації (TTFT). Сам процес інференсу сотень токенів на серверних кластерах виконується за мілісекунди, тоді як домашній процесор генерує цей самий обсяг тексту послідовно протягом 15–20 секунд.

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

Відповідь: Дані покидають периметр компанії і можуть логуватися або використовуватися для донавчання моделі провайдера, що порушує вимоги GDPR, HIPAA або захисту комерційної таємниці. У продакшні це вирішують підписанням корпоративних угод BAA/DPA (Enterprise-тарифи без збереження логів), маскуванням персональних даних (PII anonymization) перед відправкою або розгортанням локальних моделей на власних серверах.

5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

Відповідь: Rate limit — це програмне обмеження на кількість запитів (RPM/RPS) або токенів (TPM), яке сервер дозволяє надіслати за одиницю часу для запобігання перевантаженню. Його обходять чергами повідомлень із затримками, алгоритмом експоненційного відступу (Exponential Backoff з Retry), локальним кешуванням частих відповідей та ротацією пулу API-ключів чи провайдерів.

Висновок за завданням 3

Локальну модель варто обирати, коли критичною є абсолютна конфіденційність даних (медичні карти, банківська таємниця, внутрішній код компанії), автономність системи без доступу до інтернету (edge-пристрої, польові умови) та відсутність операційних витрат на платні підписки й токени. А от хмарну модель слід обирати, коли на першому місці стоїть якість формулювань, робота зі складною технічною термінологією, швидкість генерації для кінцевого користувача та відсутність потреби підтримувати власне дороге обчислювальне залізо.

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Виконано запити до локальної та хмарної моделі по три промпти різної довжини для вимірювання швидкості TTFT та швидкості генерації. А також локальні запити на холодний та теплий старт.

Таблиця 7: Результати вимірювання швидкості

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	0.078	1.051	27.4
локальна	llama3.2:3b	середній	0.079	5.218	57.3
локальна	llama3.2:3b	довгий	0.135	35.742	50.0
хмарна	gemini-3.5- flash-lite	короткий	0.772	0.852	30.5
хмарна	gemini-3.5- flash-lite	середній	0.755	1.455	368.4
хмарна	gemini-3.5- flash-lite	довгий	0.778	3.998	664.1

Таблиця 8: Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
TTFT, с	3.875	0.096
Всього, с	4.598	0.942

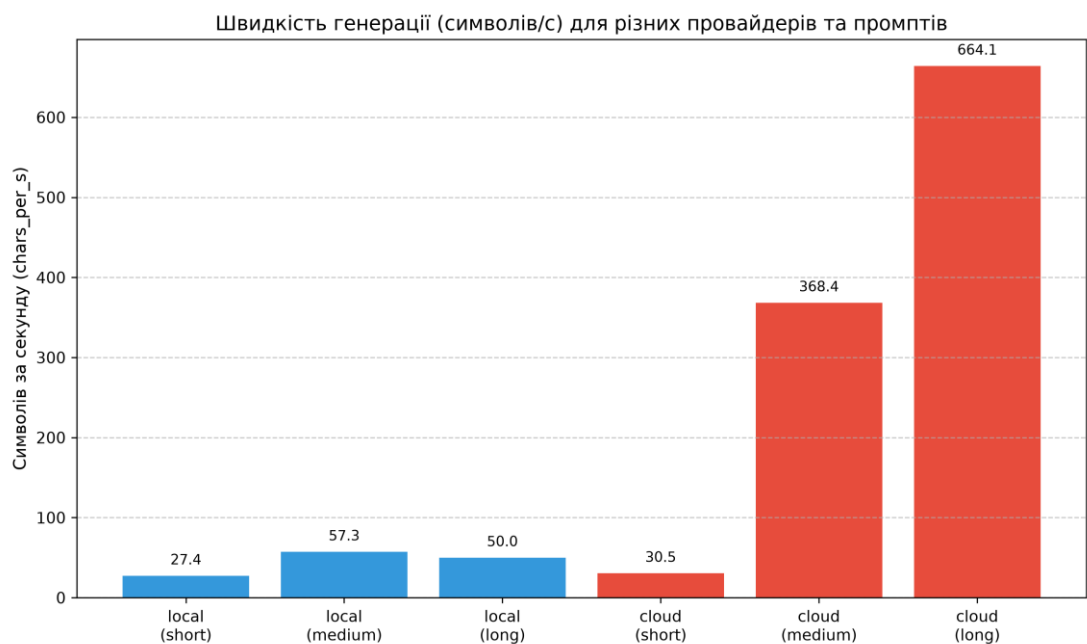


Рисунок 1 – Стовпчикова діаграма швидкості генерації символів/с

Відповіді на питання (крок 6)

1. У якій моделі кращий TTFT і чому саме в неї?

Відповідь: Кращий TTFT у локальній моделі (~ 0.09 с проти ~ 0.77 с у хмарній). Локальний процес Ollama працює безпосередньо в системній пам'яті через localhost, тоді як хмарний запит витрачає 700–800 мс на мережевий RTT (встановлення TLS-з'єднання, передачу пакетів до віддаленого дата-центру та проходження черги запитів API).

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні — поясніть.

Відповідь: Ні, вони прямо протилежні. Локальна модель має миттєвий відгук (низький TTFT), але обмежена повільною генерацією на CPU (~ 50 симв./с). Хмарна модель повільніше видає перший токен через мережу, але за рахунок серверних GPU генерує текст зі швидкістю до 664 симв./с.

3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого пром프트у до довгого?

Відповідь: На короткому пром프트і загальний час виконання був майже однаковим (1.05 с проти 0.85 с). Проте на довгому пром프트і хмарна модель відпрацювала за 4.0 с, тоді як локальна витратила майже 36 секунд (хмарна виявилася швидшою майже в 9 разів).

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Відповідь: Повтори різняться в межах 5–15% через коливання навантаження на процесор та затримки мережевих пакетів. Одного запуску недостатньо, оскільки випадковий мережевий стрибок або затримка CPU можуть спотворити метрику, тому медіана трьох прогонів дає значно стабільніший результат.

5. Чому холодний старт повільніший за теплий?

Під час холодного старту операційній системі необхідно зчитати ваги моделі (залежить від моделі та кількості мільярдів параметрів) з накопичувача у RAM, провести ініціалізацію обчислювального графу в пам'яті та прогріти кеш. При

теплому старті модель уже утримується в резидентній пам'яті, тому інференс починається миттєво.

Відповіді на питання (крок 7)

1. Що фізично відбувається під час TTFT — часу до першого токена — і чому цей показник вимірюють окремо від загальної швидкості генерації?

Відповідь: Під час TTFT виконується токенизація промпту, завантаження контексту, обчислення KV-кешу для вхідної послідовності (Prefill/Prompt evaluation) та передача першого згенерованого токена клієнту. Цей показник вимірюють окремо, бо він визначає суб'єктивну чуйність інтерфейсу для користувача.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

Відповідь: TTFT хмарної моделі збільшується через мережевий оверхед (маршрутизація, рукостискання TLS, очікування в черзі шлюзу API). Проте сама фаза генерації виконується на потужних тензорних ядрах GPU з пам'яттю HBM (пропускна здатність 2–3 ТБ/с), що в десятки разів перевершує звичайну RAM комп'ютера.

3. Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?

Відповідь: Холодний старт містить дисковий I/O: завантаження файлу моделі формату GGUF з SSD у пам'ять, алокацію буферів пам'яті та перший прогін без попереднього кешування.

4. Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?

Відповідь: Медіана стійка до викидів (outliers). Арифметичне середнє може сильно зміститися через одиничний мережевий лаг або фонову системну задачу, тоді як медіана відсікає аномальні значення.

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Відповідь: Обробка вхідного промπτу (Prefill) є паралельною: усі токени запиту обробляються одночасно через множення матриць. Генерація відповіді (Decode) є суворо послідовною (a autoregressive): кожен наступний токен вимагає повного проходу через усі шари моделі та залежить від попереднього.

Висновок за завданням 4

Локальна модель на прогрітому кеші демонструє мінімальну затримку до першого токена (TTFT ~0.08–0.1с), що забезпечує миттєву реакцію інтерфейсу. Хмарна модель через мережевий оверхед має постійний поріг затримки ~0.75–0.78с незалежно від довжини промπτу.

Хмарна модель Gemini кардинально перевершує локальну за швидкістю декодування токенів (до 664 симв./с проти 50 симв./с на довгому промπτі). Завдяки цьому загальний час генерації довгої інструкції в хмарі виявився майже в 9 разів меншим (4.0 с проти 35.7 с).

Перший виклик моделі після вивантаження з пам'яті вимагає майже 4 секунди лише на зчитування ваг з диска в оперативну пам'ять та ініціалізацію контексту, після чого модель переходить у теплий стан з миттєвим стартом.

Завдання 5. Оцінювання якості моделей за рубрикою

Виконано запити до двох локальних моделей та однієї хмарної для оцінки якості відповідей за різними тестовими завданнями.

Таблиця 13: Оцінка якості за рубрикою

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	0	0	2
2. Формат	2	2	2
3. Стиснення	0	2	2
4. Код	0	1	1

5. Логіка	0	2	2
6. Українська	0	1	2
Разом /12	2	8	11

Таблиця 14: Розбіжність оцінок між двома прогонами (так / ні)

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	ні	ні	ні
2. Формат	ні	ні	ні
3. Стиснення	ні	ні	ні
4. Код	ні	ні	ні
5. Логіка	ні	ні	ні
6. Українська	ні	ні	ні

Відповіді на питання (крок 6)

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

Відповідь: Найбільше балів набрала хмарна модель gemini-3.5-flash-lite (11 із 12), а основна різниця з'явилася на завданнях із фактчекінгу, багатокрокової фінансової арифметики та якості української мови. Локальна Llama3.2:3b повністю провалила логіку, код і факти, тоді як qwen3:4b показала високий результат на міркуваннях, але поступилася у знанні специфічних українських фактів та чистоті мови.

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

Відповідь: За підсумковою рубрикою бали між прогонами залишилися стабільними (розбіжностей за 2-бальною шкалою не зафіксовано), проте якість окремих токенів суттєво коливалася (наприклад, Llama видала 1992 і 1972 рік, а Qwen у другій спробі відняла тіло депозиту в коді). Це свідчить про те, що при ненульовій температурі генерації моделі бракує внутрішньої детермінованості, тому для критичних задач однаковість виводу потребує зниження температури до 0.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Відповідь: Так, вимогу суворо виконали всі моделі в окремих прогонах: llama3.2:3b та qwen3:4b видали чистий JSON-рядок без markdown-лапок ```json, а хмарна Gemini також повернула валідний «голий» JSON без жодних блоків коду.

4. Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?

Відповідь: У моделі llama3.2:3b проблеми виявилися катастрофічними – вона використовувала ламаний суржик («річного стовпчика», «понадаток»), неіснуючі терміни («акціонерна ділянка», «витязь акцій»), вкраплення англійської (changes, part-ласнією) і навіть несподівано згенерувала польський текст на Haskell. У qwen3:4b мова була набагато чистішою, але траплялися окремі кальки та граматичні огріхи («ділення прибутку», «випусковий суб'єкт», «різикованих»).

Приклад провалу (крок 7)

Модель: llama3.2:3b;

Завдання: 4 Код;

Повний текст запиту: compound_interest(principal, rate, years). Лише код.;

Повний текст відповіді:

Aplikacja do obliczania pożyczek z uwzględnieniem compounds interest:

```
```haskell
```

```
compounds_interest principal rate years = let factor = (1 + (rate / 100)) ^ years
in principal * factor
```

W tym kodzie, principal to wartość pieniądza, rate to stopień połowiczny procentowy, a years to ilość lat, po jakich okresie pieniędzy zostaną wypłacone.

**Правильна відповідь:**

```
```python
```

```
def compound_interest(principal, rate, years):
    return principal * (1 + rate / 100) ** years
```

Пояснення чому так сталося: Промпт складався виключно з англійських назв функцій та аргументів, після чого йшла коротка інструкція українською «Лише

код». Малій моделі на 3B параметрів забракло уваги на україномовне обмеження в кінці, через що спрацювали асоціативні зв'язки слов'янської мовної групи навчального датасету – модель випадково вирішила переключитися на польську мову. Також запит не вказував явно мову Python, тому модель вирішила обрати функціональну мову Haskell. Фраза «Лише код» була повністю проігнорована через RLHF-звичку асистента завжди ввічливо пояснювати призначення коду та описувати аргументи користувачеві.

Відповіді на питання (крок 8)

1. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

Відповідь:

- Автоматичні метрики зіставлення: ROUGE, BLEU (для сумаризації та перекладу), які порівнюють перетин слів з еталоном, або семантичні метрики (BERTScore);
- Стандартизовані академічні бенчмарки: MMLU (загальні знання), GSM8K / MATH (математичне мислення), HumanEval / MBPP (генерація та виконання коду), MT-Bench (діалогова якість);
- Arena-тестування (LMSYS Chatbot Arena): Слепіе A/B-тестування людьми, де моделі змагаються за системою рейтингів Ело (Elo rating);
- Автоматизований LLM-as-a-Judge: Використання великих флагманських моделей (GPT-4o, Claude 3.5 Sonnet) для парного порівняння відповідей із деталізованою шкалою та логуванням обґрунтування через Structured Outputs.

2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?

Відповідь: Під час етапу SFT (Supervised Fine-Tuning) та вирівнювання через RLHF моделі тренують на сотнях тисяч діалогів, де хорошим тоном вважається естетичне оформлення коду в блоки ````json ... ````. Цей патерн стає настільки сильно закріпленою поведінкою (strong prior), що легке текстове заперечення «без markdown» у промпті програє потужним внутрішнім вагам уваги на синтаксис

розмітки. Крім того, негативні інструкції («не роби X») когнітивно важче обробляються моделями, оскільки слово «markdown» саме по собі активує токени форматування.

3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?

Відповідь: Англійська мова становить 85–90% датасетів попереднього навчання, тоді як українська — значно менше 1%. Для англійської мови типове слово кодується одним токеном. Для кирилиці токенизатори малих моделей часто дроблять одне слово на 2–4 фрагменти або навіть окремі байти, що різко скорочує ефективну довжину контекстного вікна та ускладнює підтримку граматичних зв'язків. Моделі на 3 млрд параметрів недостатньо вільних ваг, щоб паралельно зі світовими знаннями зберігати специфічну морфологію, відмінки та технічні словники низькоресурсних мов, що призводить до калькування з російської чи польської.

4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?

Відповідь: Це прямий наслідок стохастичного семплування. Коли параметр температури більший за нуль ($temperature > 0$), модель не просто вибирає найімовірніший токен (Greedy search), а проводить ймовірнісну вибірку з розподілу токенів через механізми top-p (Nucleus) або top-k. Якщо на перших кроках випадково випаде інший токен, він додається у власний контекст генерації й кардинально змінює всю подальшу траєкторію думки (ефект метелика в авторегресійній моделі).

5. Що таке галюцинація мовної моделі і чому модель радше вигідає факт, ніж визнає незнання?

Відповідь: Галюцинація — це генерація семантично правдоподібного, але фактологічно хибного або неіснуючого твердження. Модель є ймовірнісним механізмом продовження послідовності токенів, у якого немає окремої бази перевірки фактів чи вбудованого модуля «самосвідомості/впевненості». Під час попереднього навчання та RLHF модель заохочують генерувати зв'язні, граматично правильні та вичерпні відповіді. Фраза «Я не знаю» під час навчання трапляється

рідко, тому модель просто обирає статистично найбільш підходящі за стилем слова, навіть якщо вони є вигадкою.

Завдання 6. Керування випадковістю: temperature і top_p

Дослідили вплив temperature та top_p різних значень при питаннях – творчому та фактичному.

Таблиця 16: Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.0	1	0
temperature	0.7	5	—
temperature	1.5	5	0
top_p	0.1	1	—
top_p	0.5	5	—
top_p	1.0	5	—

Таблиця 17: Перевірка відтворюваності

Показник	Значення
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1 (модель стабільно видала «2011»)
Усі відповіді однакові (так / ні)	так

Відповіді на питання (крок 7)

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Відповідь: Для генерації JSON слід встановити $\text{temperature} = 0.0$ (або максимум 0.1–0.2). Це мінімізує ймовірність синтаксичних помилок, виключає галюцинації у назвах ключів і гарантує суворе дотримання формату схеми.

2. Яке — генератору варіантів заголовків?

Відповідь: Для генератора заголовків оптимальним є значення $\text{temperature} = 0.7$ –0.9. Такий діапазон забезпечує високу лексичну різноманітність та креативність ідей, не скочуючись у незв'язний хаос.

3. Чим вплив top_p відрізняється від впливу temperature за характером, а не за силою?

Відповідь: temperature змінює саму форму розподілу ймовірностей (згладжує або загострює шанси всіх токенів у словнику), тоді як top_p жорстко відсікає динамічний «хвіст» малоїмовірних токенів, залишаючи для вибірки лише компактне ядро найбільш підходящих слів.

4. Чи означає $\text{temperature}=0$ повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?

Відповідь: Теоретично так, і експеримент це підтвердив: усі 10 запусків при $\text{temperature}=0.0$ повернули абсолютно однаковий результат (2011). Проте в розподілених системах із паралельними обчисленнями на GPU мінімальна недетермінованість інколи може виникати через особливості операцій з плаваючою комою (floating-point non-determinism).

5. Наведіть найбезглуздішу відповідь, яку модель видала при $\text{temperature}=1.5$, і поясніть, чому вона взагалі могла з'явитися.

Відповідь: Найбільш деградованою відповіддю стало нескінченне циклічне повторення фрази «Вася Тренувальник», перемішане з вигаданими словами («найbardівнішу», «апеккувальс») та незрозумілими символами. При $t=1.5$ розподіл настільки сплющився, що рідкісні токени отримали неприродно високу ймовірність, а згенерований фрагмент замкнув контекст моделі у самопідсилювану петлю уваги.

Відповіді на питання (крок 8)

1. Що саме математично робить параметр temperature із розподілом ймовірностей наступного токена?

Відповідь: У формулі Softmax логіти z_i діляться на значення температури T :

$$P(w_i) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

Коли $T \rightarrow 0$, різниця між найбільшим логітом та іншими прямує до нескінченності, перетворюючи Softmax на детерміновану функцію argmax . Коли $T > 1$, поділ зменшує розрив між логітами, роблячи ймовірності всіх слів у словнику більш рівномірними (ентропія зростає).

2. Чим top_p відрізняється від temperature — обидва впливають на різноманітність, але за яким механізмом?

Відповідь: Температура масштабує значення всередині експоненти для всього словника, підвищуючи шанси абсолютно будь-якого малоїмовірного слова. top_p (Nucleus sampling) сортує токени за спаданням і накопичує їхню сумарну ймовірність до порогу p (наприклад, 0.9), повністю відкидаючи решту токенів. Таким чином, top_p адаптивно звужує вибірку в ситуаціях, де наступне слово очевидне, і розширює її там, де є багато рівноцінних варіантів.

3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?

Відповідь: Розробники OpenAI та автори більшості досліджень рекомендують фіксувати один із параметрів (наприклад, тримати $\text{top}_p = 1.0$ і регулювати лише temperature, або навпаки). Одночасна зміна ускладнює контроль за поведінкою моделі, оскільки вони взаємно накладаються: низький top_p може повністю нівелювати підвищення температури, просто відрізавши збагачений хвіст розподілу.

4. Чому навіть при temperature=0 мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?

Відповідь: На апаратному рівні сучасні GPU виконують операції матричного множення паралельно сотнями потоків. Через зміну порядку додавання чисел із рухомою комою (а додавання у float32/bfloat16 не є асоціативним: $(a+b)+c \neq a+(b+c)$) виникають похибки округлення в молодших бітах. Якщо два провідні токени мають майже однакові логіти, ця мікроскопічна різниця може змінити переможця, що веде до іншої траєкторії всієї подальшої генерації.

5. Які значення temperature використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?

Відповідь: Для витягу структурованих даних (JSON, SQL, аналіз документів, класифікація, виклик інструментів) використовують значення temperature = 0.0–0.2 для досягнення стабільності й валідності синтаксису. Для написання креативних статей, мозкового штурму, художніх текстів та створення ігрових діалогів застосовують temperature = 0.7–0.9.

Висновок за завданням 6

Для наступних лабораторних робіт, де від моделі вимагається повернення суворо структурованих даних (JSON, об'єкти, схеми), я оберу параметри temperature = 0.0 та top_p = 1.0. Налаштування «за замовчуванням» (temperature = 0.7–1.0) орієнтовані на звичайні чат-боти для створення «живого» різноманітного діалогу. Для структурованого виводу дефолтна температура є непридатною: вона підвищує ризик синтаксичного зламу JSON (пропущені лапки, коми), додавання зайвих вступних слів або спотворення назв полів схеми, що призведе до збоїв парсерів у коді.

Підсумки

Заповнили підсумкову порівняльну таблицю моделей які було розглянуто та протестовано у процесі виконання лабораторної роботи.

Таблиця 18: Підсумкове порівняння моделей

Критерій	Локальна мала (3В)	Локальна більша (4В)	Хмарна
Швидкість генерації	Середня на CPU (27-57 симв./с)	Низька на CPU (через генерацію довгих токенів міркувань)	Дуже висока (300-600 симв./с на GPU в дата центрах)
TTFT	Мінімальний (0,08-0,13с)	Низький (0,1-0,3с)	Помірний (0,75-0,78с через мережеву затримку)
Якість, балів /12	2/12 (часті галюцинації, злам мови)	8/12 (висока точність міркувань, є мовні недоліки)	11/12 (відмінна логіка, мова та факти)
Стабільність формату	Висока	Висока	Висока
Потрібні ресурси	3-5Гб RAM, може цілком комфортно працювати на звичайному сучасному CPU	4-6Гб RAM, значне навантаження на CPU, доводиться пристойно так чекати	Власне залізо не має значення, потрібне лиш стабільне мережеве підключення
Вартість	Безкоштовно (витрати лише на світло при постійному обороті пристрою з моделю)	Безкоштовно	Pay-as-you-go за токени (або безкоштовний лімітний тариф API)
Куди йдуть ваші дані	Залишаються виключно в локальному обороті	Залишаються в локальному форматі	Передаються на сервери Google
Працює без інтернету	Так	Так	Ні
Коли обирати саме її	Коли потрібен миттєвий відгук інтерфейсу (тобто низький TTFT), повна автономність без мережі та конфіденційність для прости рутинних	Коли необхідна приватність даних та потрібна обробка складної логіки чи генерації коду, а затримка не є перешкодою для користувача	Коли основними факторами є висока швидкість на великих обсягах тексту, бездоганна грамотність української, багатокрокове

	задач, де відсутні складні математичні розрахунки та мінімальна взаємодія з українською мовою		міркування та актуальні знання, а безпека даних дозволяє надсилати дані у зовнішню хмару
--	---	--	--

Загальні висновки

1) Локальна й хмарна моделі використовують єдиний інтерфейс, тому перехід між ними не потребує змін у логіці програми – це основа стратегії резервування провайдерів у реальних проєктах;

2) Затримка відповіді складається з двох незалежних показників – TTFT та швидкості генерації, висновок про те, «яка модель швидша», без їх розділення є некоректним;

3) Режим міркування є інструментом із чіткою областю застосування: він окупається на багатокроковому виведенні й є чистою перевитратою на фактичних питаннях і переказі;

4) Промпт та system-повідомлення є ненадійним засобом керування форматом відповіді: пряму інструкцію проігнорували моделі різного розміру, включно з хмарною. Надійність досягається програмною валідацією, а не формулюванням;

5) Мовні моделі є недетермінованими системами, тому будь-яке вимірювання потребує повторів і усереднення, а порівняння двох версій вимагає проведення статистичних тестів за однакових значень параметрів генерації (temperature, top_p) та фіксованого контексту.